

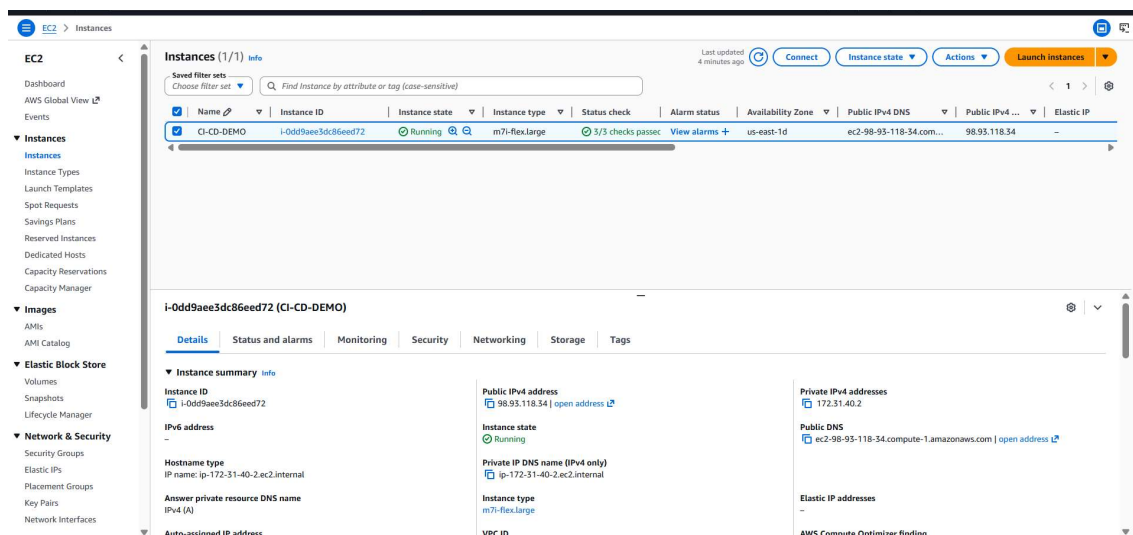
Jenkins CI-CD Pipeline with Docker and Github

Project Overview - This project automates application deployment using Jenkins, Docker, and AWS EC2. When code is pushed to GitHub, Jenkins builds and deploys the app automatically, and it runs inside a Docker container on EC2.

Technologies Used -

1. Jenkins – For automating CI/CD pipeline
2. GitHub – For source code management and webhook trigger
3. Docker – For containerizing and running the application
4. AWS EC2 – For hosting the application
5. Linux (Ubuntu) – Server environment
6. Shell Scripting (Bash) – For writing deployment logic

1. Launch ec-2 instance



Connect - `ssh -i "C:\Users\mrsoh\Downloads\new-key.pem" ubuntu@98.93.118.34`

2.install java and jenkins

Java -

sudo apt update

sudo apt install openjdk-17-jdk -y

java -version

Jenkins -

sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \

<https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key>

echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \

<https://pkg.jenkins.io/debian-stable/binary/> | sudo tee \

/etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update

sudo apt install jenkins

```
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-40-2:~$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-04-02 10:57:50 UTC; 11s ago
     Main PID: 4575 (java)
        Tasks: 52 (limit: 9279)
      Memory: 854.8M (peak: 863.4M)
         CPU: 15.645s
       CGroup: /system.slice/jenkins.service
               └─4575 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins
Apr 02 10:57:46 ip-172-31-40-2 jenkins[4575]: [LF]> This may also be found at: /var/lib/jenkins/secrets/initialAdminPas
Apr 02 10:57:46 ip-172-31-40-2 jenkins[4575]: [LF]>
Apr 02 10:57:46 ip-172-31-40-2 jenkins[4575]: [LF]> *****
Apr 02 10:57:46 ip-172-31-40-2 jenkins[4575]: [LF]> *****
Apr 02 10:57:50 ip-172-31-40-2 jenkins[4575]: 2026-04-02 10:57:50.274+0000 [id=30] INFO jenkins.InitReact
Apr 02 10:57:50 ip-172-31-40-2 jenkins[4575]: 2026-04-02 10:57:50.287+0000 [id=23] INFO hudson.lifecycle.
Apr 02 10:57:50 ip-172-31-40-2 systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Apr 02 10:57:50 ip-172-31-40-2 jenkins[4575]: 2026-04-02 10:57:50.386+0000 [id=49] INFO h.m.DownloadServic
Apr 02 10:57:50 ip-172-31-40-2 jenkins[4575]: 2026-04-02 10:57:50.389+0000 [id=49] INFO hudson.util.Retrie
lines 1-20/20 (END)
```

3.install git and docker

Git -

sudo apt update

sudo apt install git -y

git --version

Docker -

sudo apt install docker.io -y

sudo systemctl start docker

sudo systemctl enable docker

docker --version

sudo systemctl enable jenkins (Server restart hua → Jenkins automatically start)

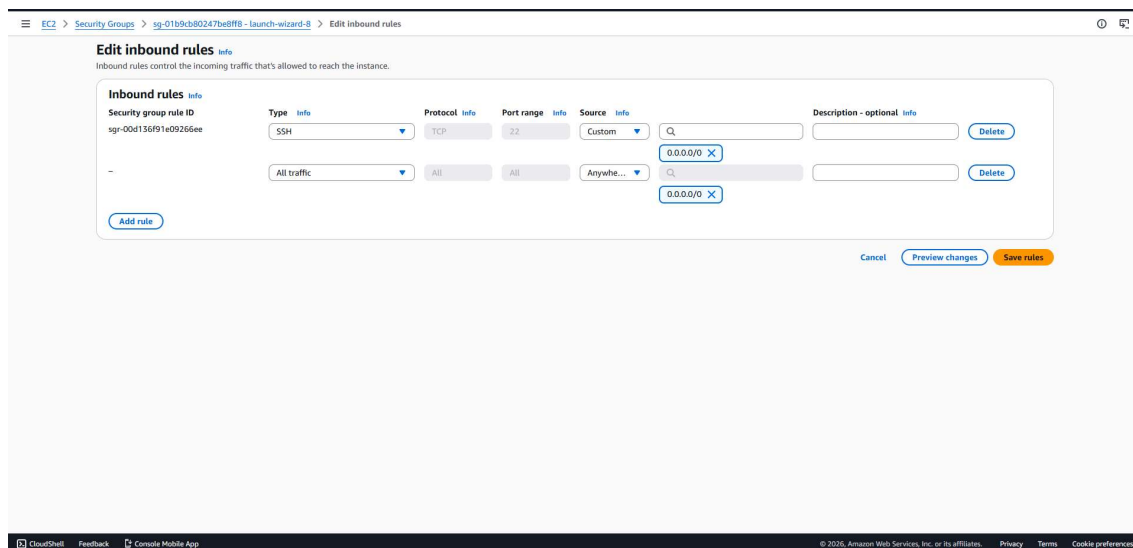
sudo systemctl enable docker (Server restart hua → Docker automatically start)

sudo usermod -aG docker jenkins (allow jenkins to interact with docker)

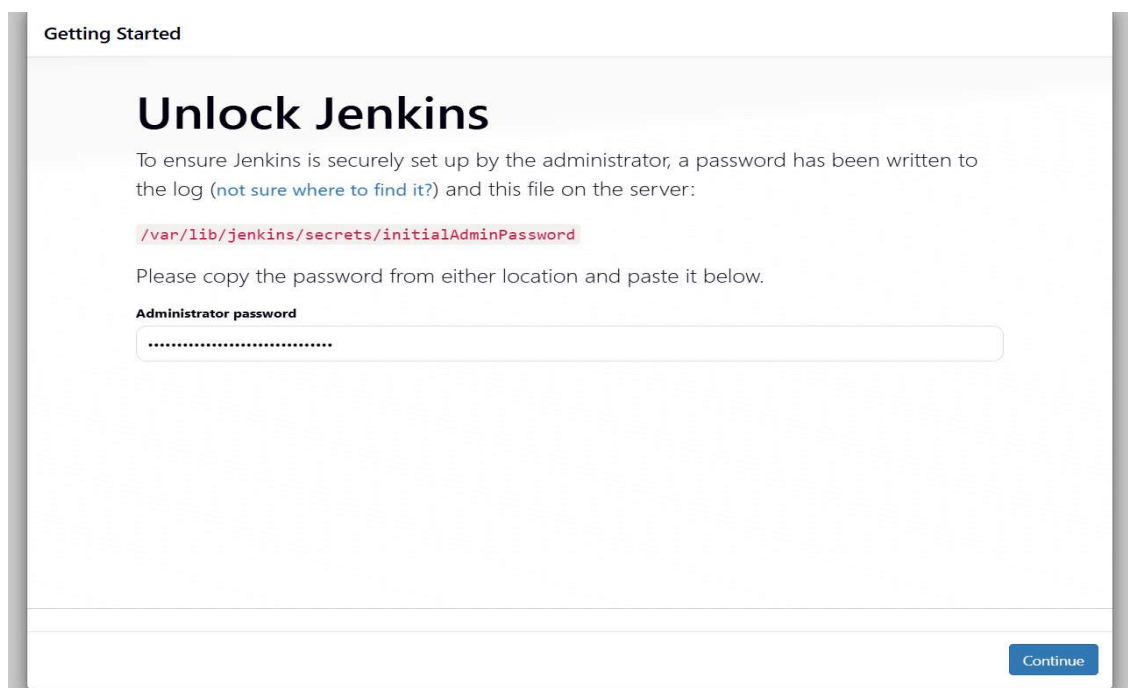
sudo systemctl restart jenkins

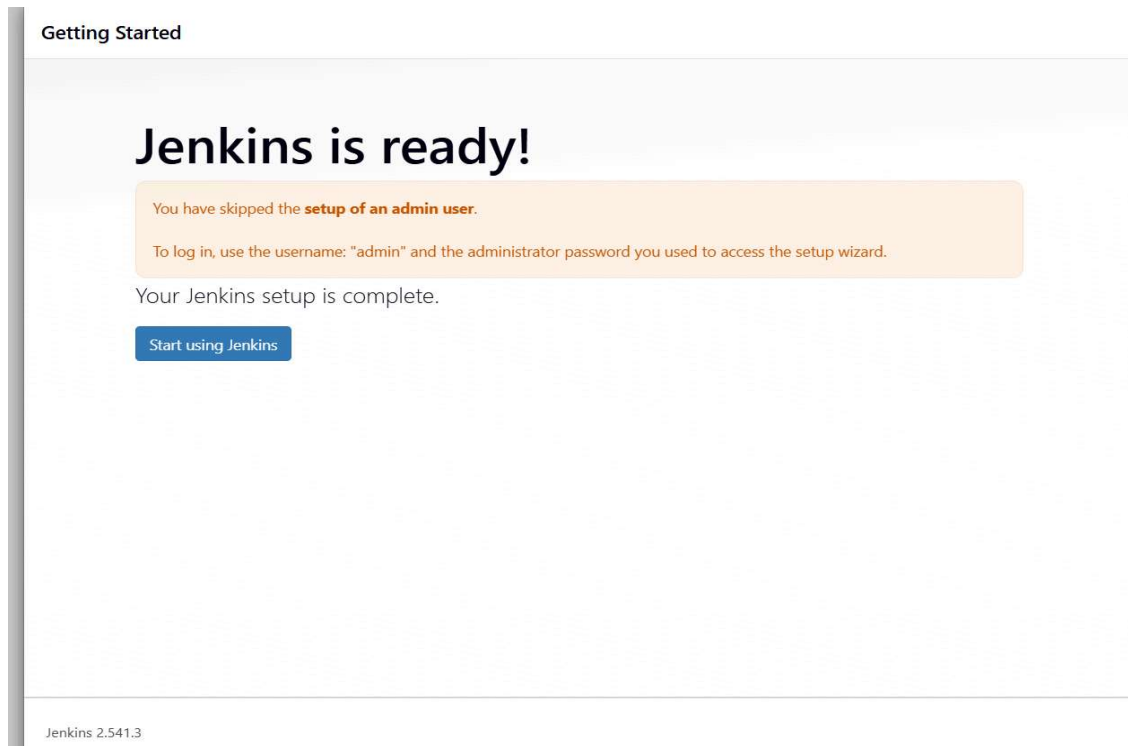
4.Before starting the jenkins edit the inbound rules and add new rule.

Ec2>>Instances>>security groups>>Inbound rules>>edit inbound rules>>add rule.



5.start the jenkins



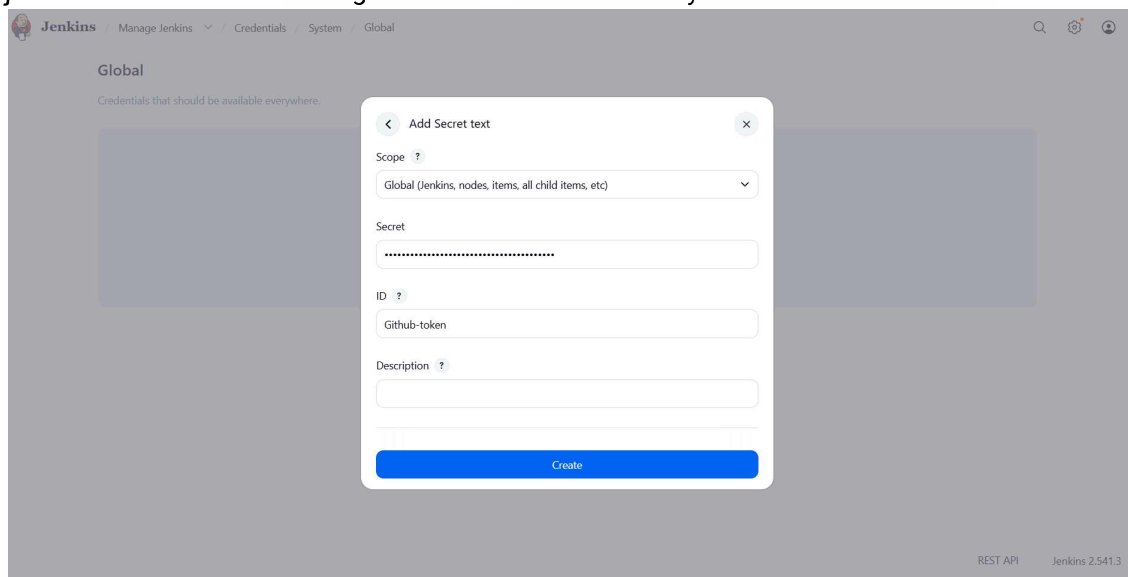


6. Create Github token

Github>>profile>>settings>>Developer-settings>>Tokens
classic>>Generate-new-token

7. Add token in jenkins

jenkins-dashboard>>Manage Jenkins>>Credentials>>system>>Global>>add text.

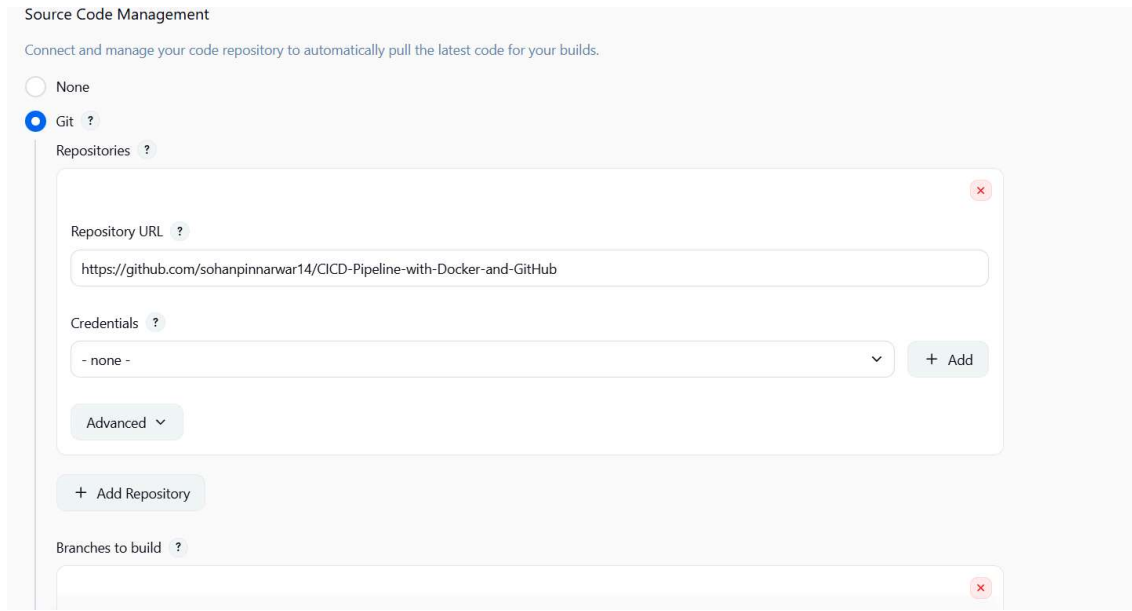


8.Create Jenkins Job

jenkins-dashboard>>create-job>>Freestyle-Project>>save.

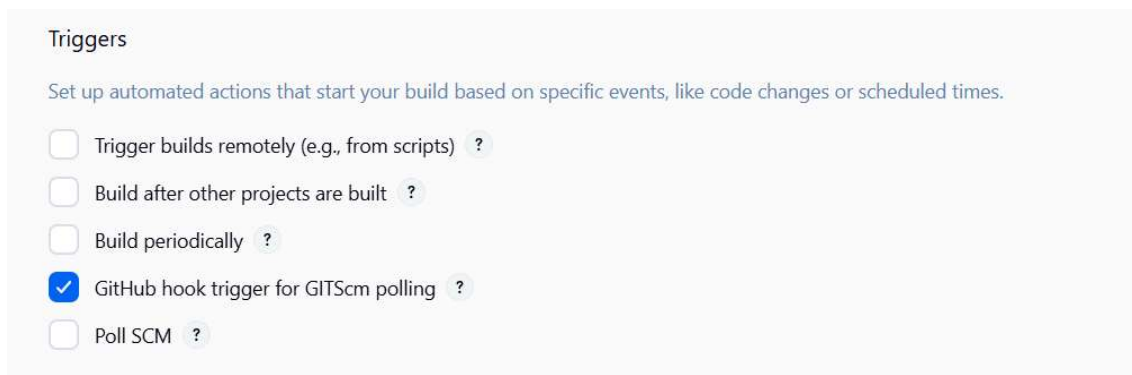
9.pipeline-demo>>source-code-management->GIT

->Add Git repo url's



The screenshot shows the 'Source Code Management' configuration page in Jenkins. At the top, it says 'Source Code Management' and 'Connect and manage your code repository to automatically pull the latest code for your builds.' Below this, there are two radio buttons: 'None' and 'Git'. The 'Git' option is selected. Under the 'Git' section, there is a 'Repositories' section with a '+' icon and a question mark. Inside this section, there is a 'Repository URL' field with the value 'https://github.com/sohanpinnarwar14/CICD-Pipeline-with-Docker-and-GitHub'. Below the URL field is a 'Credentials' dropdown menu with the value '- none -' and a '+ Add' button. There is also an 'Advanced' dropdown menu. At the bottom of the 'Repositories' section is a '+ Add Repository' button. Below the 'Repositories' section is a 'Branches to build' section with a '+' icon and a question mark.

10.Enable Webhook Trigger



The screenshot shows the 'Triggers' configuration page in Jenkins. At the top, it says 'Triggers' and 'Set up automated actions that start your build based on specific events, like code changes or scheduled times.' Below this, there are five checkboxes with labels and question marks: 'Trigger builds remotely (e.g., from scripts)', 'Build after other projects are built', 'Build periodically', 'GitHub hook trigger for GITScm polling', and 'Poll SCM'. The 'GitHub hook trigger for GITScm polling' checkbox is checked.

11. Add build steps

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute shell ?

Command

See the list of available environment variables

```
#!/bin/bash

container_id=$(docker ps --filter "status=running" --format "{{.ID}}")

if [ -n "$container_id" ]; then
  docker cp /var/lib/jenkins/workspace/devops-project/. "$container_id":usr/share/nginx/html
else
  docker build -t server /var/lib/jenkins/workspace/devops-project
  docker run -d -p 9090:80 server
fi
```

12. Setup Github webhooks

Github>>repo>>settings>>Webhooks>>add payload
http://<EC2-PUBLIC-IP>:8080/github-webhook/

Webhooks / Add webhook

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *

Content type *

Secret

13. Test Pipeline

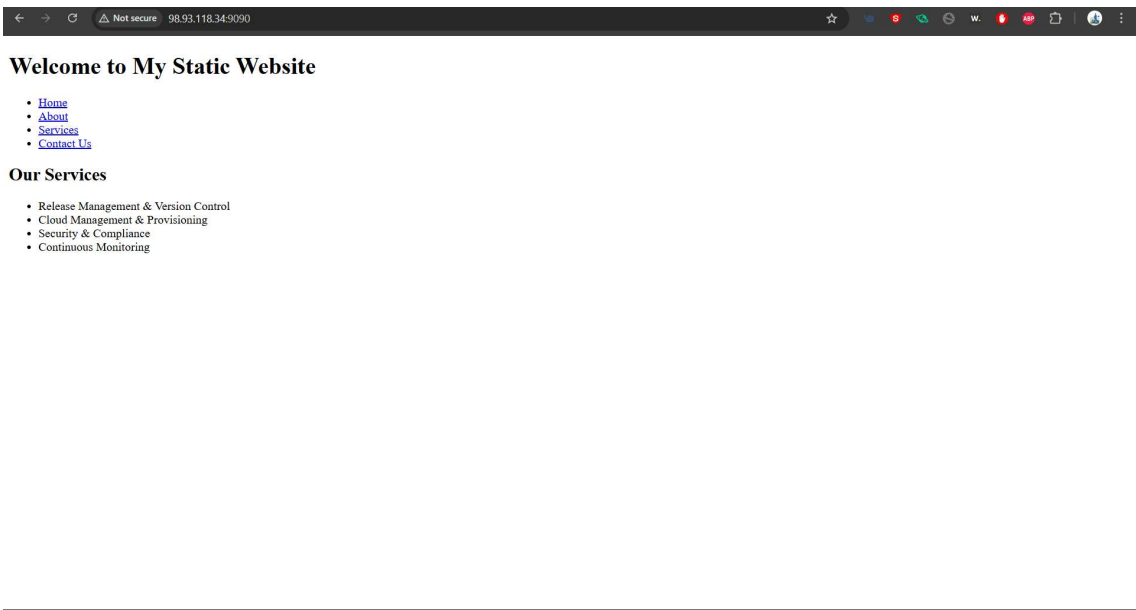
Console

DownloadCopyView as plain text

Started by user admin
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Pipeline-demo
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Pipeline-demo/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/sohanpinnarwar14/CICD-Pipeline-with-Docker-and-GitHub # timeout=10
Fetching upstream changes from https://github.com/sohanpinnarwar14/CICD-Pipeline-with-Docker-and-GitHub
> git --version # timeout=10
> git --version # 'git version 2.43.0'
> git fetch --tags --force --progress -- https://github.com/sohanpinnarwar14/CICD-Pipeline-with-Docker-and-GitHub +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision a8475f0e72b98a1151cb73a13e74251348b2bc7a (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f a8475f0e72b98a1151cb73a13e74251348b2bc7a # timeout=10
Commit message: "Update build-steps.sh"
> git rev-list --no-walk a8475f0e72b98a1151cb73a13e74251348b2bc7a # timeout=10
[Pipeline-demo] \$ /bin/bash /tmp/jenkins7226150942880736166.sh
Finished: SUCCESS

14. Test Pipeline

http://<EC2-PUBLIC-IP>:9090



Testing project -

I made changes in the application code locally and pushed them to GitHub.

```
git add .
git commit -m "updated UI"
git push origin main
```

Welcome to My Static Website -- sohan

- [Home](#)
- [About](#)
- [Services](#)
- [Contact Us](#)

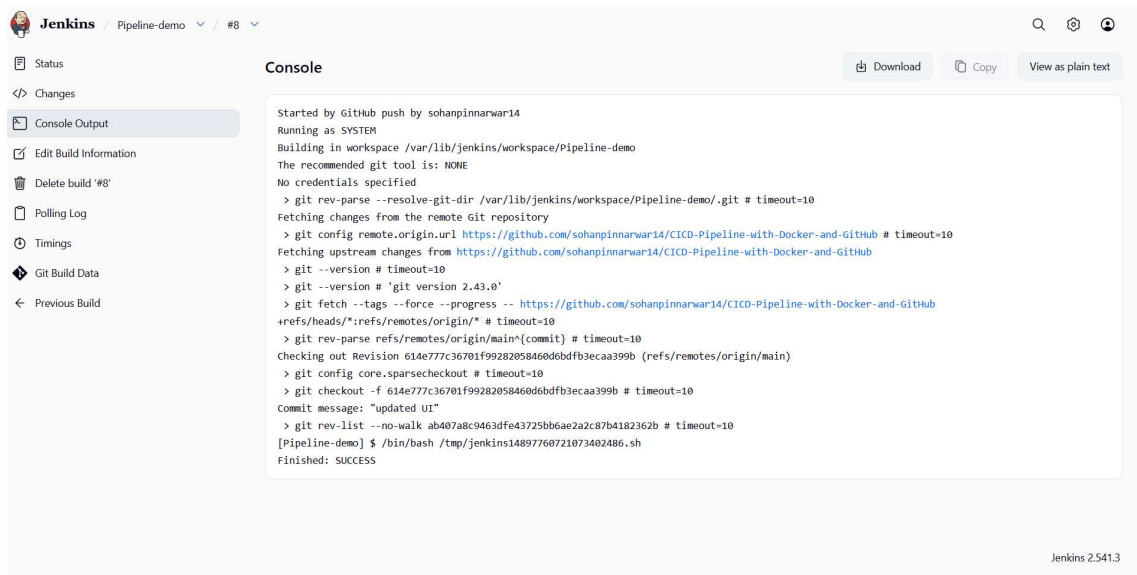
Our Services

- Release Management & Version Control
- Cloud Management & Provisioning
- Security & Compliance
- Continuous Monitoring

```
PS C:\Users\mrsoh\OneDrive\Desktop\CICD Pipeline with Docker and GitHub> git push origin main
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/sohanpinnarwar14/CICD-Pipeline-with-Docker-and-GitHub.git
   5cd64ed..ab407a8  main -> main
● PS C:\Users\mrsoh\OneDrive\Desktop\CICD Pipeline with Docker and GitHub> git add .
● PS C:\Users\mrsoh\OneDrive\Desktop\CICD Pipeline with Docker and GitHub> git commit -m "updated UI"
[main 614e777] updated UI
 1 file changed, 16 insertions(+)
● PS C:\Users\mrsoh\OneDrive\Desktop\CICD Pipeline with Docker and GitHub> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 563 bytes | 563.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/sohanpinnarwar14/CICD-Pipeline-with-Docker-and-GitHub.git
   ab407a8..614e777  main -> main
○ PS C:\Users\mrsoh\OneDrive\Desktop\CICD Pipeline with Docker and GitHub>
```

Docker container running status -

```
ubuntu@ip-172-31-40-2:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS                               NAMES
2e7e9a01d535   server    "/docker-entrypoint..." 45 minutes ago Up 45 minutes    0.0.0.0:8090->80/tcp, [::]:8090->80/tcp   musing_shtern
ubuntu@ip-172-31-40-2:~$
```

New updates.....

Welcome to My Static Website -- sohan

- [Home](#)
- [About](#)
- [Services](#)
- [Contact Us](#)

Our Services

- Release Management & Version Control
- Cloud Management & Provisioning
- Security & Compliance
- Continuous Monitoring
- Release Management & Version Control
- Cloud Management & Provisioning
- Security & Compliance
- Continuous Monitoring
- Release Management & Version Control
- Cloud Management & Provisioning
- Security & Compliance
- Continuous Monitoring
- Release Management & Version Control
- Cloud Management & Provisioning
- Security & Compliance
- Continuous Monitoring
- Release Management & Version Control
- Cloud Management & Provisioning
- Security & Compliance
- Continuous Monitoring

This triggered the Jenkins pipeline automatically via webhook. Jenkins then pulled the latest code, built the Docker image, and deployed the updated application on the EC2 instance. Finally, I verified the changes by accessing the application through the public IP and confirming that the updates were reflected successfully.

